Rockchip Developer Guide Android 3A Module

文件标识：RK-KF-YF-E74

发布版本：V1.0.0

日期：2026-03-31

文件密级：□绝密 □秘密 □内部资料 ■公开

免责声明

本文档按"现状"提供，瑞芯微电子股份有限公司（"本公司"，下同）不对本文档的任何陈述、信息和内容的准确性、可靠性、完整性、适销性、特定目的性和非侵权性提供任何明示或暗示的声明或保证。本文档仅作为使用指导的参考。

由于产品版本升级或其他原因，本文档将可能在未经任何通知的情况下，不定期进行更新或修改。

商标声明

"Rockchip"、"瑞芯微"、"瑞芯"均为本公司的注册商标，归本公司所有。

本文档可能提及的其他所有注册商标或商标，由其各自拥有者所有。

版权所有 © 2026 瑞芯微电子股份有限公司

超越合理使用范畴，非经本公司书面许可，任何单位和个人不得擅自摘抄、复制本文档内容的部分或全部，并不得以任何形式传播。

瑞芯微电子股份有限公司

Rockchip Electronics Co., Ltd.

地址： 福建省福州市铜盘路软件园A区18号

网址： www.rock-chips.com

客户服务电话： +86-4007-700-590

客户服务传真： +86-591-83951833

客户服务邮箱： fae@rock-chips.com

前言

概述

本文主要介绍Rockchip Android 3A模块，开发或技术支持人员通过阅读此文档，对Android平台3A模块有一个初步的了解，帮助阅读者开发和调试。

产品版本

芯片名称	系统版本
适配所有芯片	Android16及以上

读者对象

本文档（本指南）主要适用于以下工程师：

技术支持工程师

软件开发工程师

修订记录

版本号	作者	修改日期	修改说明
V1.0.0	刘鹏飞	2026-03-31	初始版本

目录

Rockchip Developer Guide Android 3A Module
- 说明
- 启用
  - 算法库
    - 授权
    - 编译
    - 启用模块
    - 算法参数
      - 算法参数说明
      - 算法参数调试
      - 更新算法参数
  - Android
    - 获取回采数据
      - 硬回采
      - SAI dataline 间路由
      - DLP
    - 调整音频参数
      - 修改声道参数
      - 修改 MIC 录音参数
- 验证
- Q&A
  - 如何确认需要启用哪些算法模块
  - 没有生成输出文件
  - 输出文件数据全 0
  - cannot set hw params
    - 检查通路设置

检查声卡相关
示例
确定需求
确认硬件配置
配置回采方式
修改音频参数
验证分析

说明

阅读本文档时请注意标注的红色字体之处, 切勿遗漏

本文档适用于RK 平台Android16及以上版本SDK，源码位于 `vendor/rockchip/modules/audio_aidl/modules/preprocess`

不再集成 Android Audio Effect 框架，但仍然使用 librkaudio_vqe 算法库, 将音效库命名为 rkpreprocess

算法库分为通用版本和付费版本, SDK 初始版本为通用版本, 如果需要使用付费版本请联系我司业务申请授权

模块	描述
AEC	包括 delay
BF	包括 fast_aec | aes | eq | gsc (仅适用于多 MIC) | agc | anr | nlp | dereverb | cng | dtd | howl | doa (仅适用于多 MIC) | wind | wakeup | ainr (授权付费)

	描述
MIC数量	单 MIC | 双 MIC | 多 MIC (授权定制付费)

启用

算法库

算法库详细的介绍请参考文档

授权

多麦克风算法库需要付费授权, 一般申请通过后, 回复邮件中会包含但不限于以下的授权信息:

	说明
用户	当前用户, 请勿外传
密码	密码, 请勿外传
次数	可使用次数
授权模块	audio3a

请参考文档使用授权工具进行授权, 如果是设备上直接授权必须保证设备可以联网, 如果是 PC 上授权必须保证 PC 可以联网

注意：使用 rkauth_tool_bin 和 rkauth_verify_licence 时，需要追加选项 `-m "audio3a"`

授权成功后将生成的授权信息文件 key.lic 保存到 `/data/key.lic`

参考命令：

rkauth_tool_bin --user="你的账号" --passwd="你的密码" -m "audio3a" --output="/data/key.lic"

量产时，需要对授权流程代码化，可以根据授权工具及文档提供的方法，自行完成，也可以redmine提交需求协助

audio3a算法默认是访问的/data/key.lic，如需变更key路径，算法有如下接口：

```
int rkaudio_license_path_set(char* path);
```

在`vendor/rockchip/modules/audio_aidl/modules/preprocess/rkaudio_preprocess.h`中增加此函数声明，并在`algo_init`函数开头调一下，变更key的访问路径

编译

RK 3A算法库默认编译打包进apex库`com.rockchip.hardware.audio`，且默认启用`ANR`及`AGC`功能，无需显式打开编译开关

若要关闭RK 3A算法，请修改`vendor/rockchip/modules/audio_aidl/modules/Android.bp`

```
diff --git a/modules/Android.bp b/modules/Android.bp
index 8ffe836..b88c6ef 100644
--- a/modules/Android.bp
+++ b/modules/Android.bp
@@ -105,7 +105,6 @@ cc_library {
         "-Werror",
         "-Wthread-safety",
         "-DBACKEND_NDK",
-        "-DUSE_AUDIO_PREPROCESS",   /* disable rkpreprocess: remove this flag */
     ],
 }
```

修改后可以单编

```
# 打开console，根目录下lunch目标，不需要执行后面的./build.sh
source build/envsetup.sh && lunch xxxxxxx
m com.rockchip.hardware.audio
```

安装

```
adb install
xxx\out\target\product\rk3576_a16\vendor\apex\com.rockchip.hardware.audio.apex
adb shell sync
adb shell reboot
```

若以上单编方式无效，请参考Audio Hal指引文档
`RKDocs/android/audio/Rockchip_Developer_Guide_Android_Audio_AIDL_HAL_CN.pdf`

启用模块

参考如何确认需要启用哪些算法模块

根据实际需求和当前算法库版本支持的模块, 修改
`vendor/rockchip/modules/audio_aidl/modules/preprocess/rkaudio_effect_preprocess.json` 中对应模块的 `status` 为 `enable` 来启用模块

模块的开关状态说明如下:

status	说明
enable	启用
disable	关闭

模块的依赖关系说明如下:

模块	依赖模块
aec	无
delay	aec
bf	无
fast_aec, aes, eq, gsc, agc, anr, nlp, dereverb, cng, dtd, howl, doa, wind, wakeup, ainr	bf

模块之间存在依赖关系时, 需要先启用依赖的模块才能启用某个模块

算法参数

算法库的默认参数配置保存在 rkaudio_effect_preprocess.json，默认保存到 SDK 的
`vendor/rockchip/modules/audio_aidl/modules/preprocess/` 目录

出厂默认安装在设备的 `/vendor/etc/` 目录

算法参数说明

参数说明请参考文档

算法参数调试

确保固件版本是 userdebug, 确保 PC 可以通过 ADB 与设备建立连接

在 PC 端执行以下两条指令:

```
adb root
adb remount
```

初次烧录完固件需要在执行上述两条指令后重启, 再重新执行上述两条指令

更新算法参数

修改 rkaudio_effect_preprocess.json, 并手动 adb push 到设备对应的目录

```
adb push rkaudio_effect_preprocess.json /vendor/etc/
```

Android

获取回采数据

若使能aec/fast_aec/aes模块，需要将回采数据和MIC 数据绑定为同一声卡，根据项目实际的硬件设计，主要有以下3种方式获取回采数据

硬回采

[L1][R1]... 表示 MIC 数据; [L2][R2]... 表示回采数据, 在 SPK 的前级回环到某个音频接口 (CPU DAI1)

```
HAL                 | Kernel                              | Hardware
                    |                                     |
+-----------------+ | [L2][R2]...          +----------+ |               +-----+
| Output Stream   *-------------------------->* CPU DAI0 *----.------------>* SPK |
+-----------------+ |                      +----------+ |  \            +-----+
                    |                                   |  |
                    |                                   |  |(Wired)
"preproc_with_ref_interleaved"                          |  |
+-----------------+ | [L1][R1][L2][R2]...  +----------+ |  v [L1][R1]...+-----+
| Input Stream    *<==========================* CPU DAI1 *<====+<-----------* MIC |
+-----------------+ |                      +----------+ |               +-----+
                    |                                   |
                    |                                   |
```

```
HAL                 | Kernel                              | Hardware
                    |                                     |
+-----------------+ | [L2][R2]...          +----------+ |               +-----+
| Output Stream   *-------------------------->* CPU DAI0 *----.------------>* SPK |
+-----------------+ |                      +----------+ |  \            +-----+
                    |                                   |  |
                    |                     +-------------+ |  |(Wired)
                    |                     | Combo DAI   | |  |
                    |                     |+----------+| |  /
"preproc_with_ref_interleaved"            || CPU DAI1 *<---'
+-----------------+ | [L1][R1][L2][R2]... |+----------+| |
| Input Stream    *<========================*            | |
+-----------------+ |                     |+----------+| |    [L1][R1]...+-----+
                    |                     || CPU DAI2 *<-----------------* MIC |
                    |                     |+----------+| |               +-----+
                    |                     +-------------+ |
```

硬件回采CPU dai的dts如何配置，请参考文档，没有硬件回采则考虑以下两种方式实现软回采

SAI dataline 间路由

如果 CPU DAI 使用的是 SAI (sai0, sai1 ...), 可以考虑使用 SAI dataline 间路由配置, 回环某个使用的 SDO 到某个空闲的 SDI

[L1][R1]... 表示 MIC 数据; [L2][R2]... 表示回采数据; SDOx 表示正在使用的 SDO; SDIx 表示正在使用的 SDI; SDIy 表示空闲的 SDI, 回环来自 SDOx 的数据

```
HAL                 | Kernel                              | Hardware
                    |                                     |
                    |                         +----------+  |
+-----------------+  | [L2][R2]...             | CPU DAI0 |  |  (SDOx)       +-----+
| Output Stream  *-------------------------*--.--------*-----------------*>* SPK |
+-----------------+  |                         |   \      |  |               +-----+
                    |                         |   |      |  |
                    |                         |   |      |  |
"preproc_with_ref_interleaved"                  |   |(SDIy)|  |  (SDIx)
+-----------------+  | [L1][R1][L2][R2]...     |   v      |  |  [L1][R1]...  +-----+
| Input Stream   *<=========================*===+<-----*------------------* MIC |
+-----------------+  |                         |          |  |               +-----+
                    |                         +----------+  |
                    |                                     |
```

SDI 回环源

`sound/soc/rockchip/rockchip_sai.c`

```
static const char * const lpx_text[] = {
    "From SDO0", "From SDO1", "From SDO2", "From SDO3" };

static SOC_ENUM_SINGLE_DECL(lp3_enum, SAI_PATH_SEL, 28, lpx_text);
static SOC_ENUM_SINGLE_DECL(lp2_enum, SAI_PATH_SEL, 26, lpx_text);
static SOC_ENUM_SINGLE_DECL(lp1_enum, SAI_PATH_SEL, 24, lpx_text);
static SOC_ENUM_SINGLE_DECL(lp0_enum, SAI_PATH_SEL, 22, lpx_text);

static const struct snd_kcontrol_new rockchip_sai_controls[] = {
    // ...
    SOC_ENUM("SDI3 Loopback Src Select", lp3_enum),
    SOC_ENUM("SDI2 Loopback Src Select", lp2_enum),
    SOC_ENUM("SDI1 Loopback Src Select", lp1_enum),
    SOC_ENUM("SDI0 Loopback Src Select", lp0_enum),
    // ...
};
```

SDI 回环开关

`sound/soc/rockchip/rockchip_sai.c`

```
static const char * const lps_text[] = { "Disable", "Enable" };

static SOC_ENUM_SINGLE_DECL(lp3_switch, SAI_PATH_SEL, 21, lps_text);
static SOC_ENUM_SINGLE_DECL(lp2_switch, SAI_PATH_SEL, 20, lps_text);
static SOC_ENUM_SINGLE_DECL(lp1_switch, SAI_PATH_SEL, 19, lps_text);
```

```
static SOC_ENUM_SINGLE_DECL(lp0_switch, SAI_PATH_SEL, 18, lps_text);

static const struct snd_kcontrol_new rockchip_sai_controls[] = {
    // ...
    SOC_ENUM("SDI3 Loopback Switch", lp3_switch),
    SOC_ENUM("SDI2 Loopback Switch", lp2_switch),
    SOC_ENUM("SDI1 Loopback Switch", lp1_switch),
    SOC_ENUM("SDI0 Loopback Switch", lp0_switch),
    // ...
};
```

比如 2 channel mic + 2 channel 回采, 使用 SDO0 输出到 SPK, SDI0 接收来自 MIC 的数据, 可以从空闲的 SDI1, SDI2 和 SDI3 中选择一个回环来自 SDO0 的数据, 比如选择 SDI1 回环来自 SDO0 的数据, 则可以参考下面修改:

修改通路配置, 增加在对应的通路配置xml文件中

不同的声卡名对应不同通路配置文件, 通过 `vendor/rockchip/modules/audio_aidl/config/platform/audio_rockchip_platform_config.xml` 确认, 没有匹配则使用默认的mixer_paths_default.xml

```
diff --git a/config/mixer-path/mixer_paths_default.xml b/config/mixer-
path/mixer_paths_default.xml
index 23a7860..59cd875 100644
--- a/config/mixer-path/mixer_paths_default.xml
+++ b/config/mixer-path/mixer_paths_default.xml
@@ -117,6 +117,8 @@
     <!-- In Builtin MIC -->
     <path name="in_builtin_mic normal">
         <ctl name="Capture MIC Path" value="Main Mic"/>
+        <ctl name="SAI1 SDI1 Loopback Src Select" value="From SDO0"/>
+        <ctl name="SAI1 SDI1 Loopback Switch" value="Enable"/>
     </path>

     <!-- In Wired Headset -->
```

注: 因内核版本差异, 实际的控件名称可能会不同, 可以通过 tinymix 查看

DLP

若CPU dai没有采用SAI，那么软回采只能采用DLP方式

[L1][R1]... 表示 MIC 数据; [L2][R2]... 表示回采数据, 缓存到 DLP



```
| Input Stream   *<=========================*====+<----*------------------* MIC |
+----------------+  |                       |          |  |               +-----+
                    |                       +----------+  |
                    |                                     |
```

修改 kernel 配置, 编译 DLP

```
CONFIG_SND_SOC_ROCKCHIP_DLP_PCM
```

修改 dts 配置, 在对应的 cpu dai 上使能 DLP

```
&i2s {
        // ...
        rockchip,digital-loopback;
        // ...
};
```

> cpu dai 可以是: i2s0_8ch, i2s1_8ch, i2s2_2ch, i2s3_2ch, sai0, sai1 等等

修改通路配置, 增加在对应的通路配置xml文件中

不同的声卡名对应不同通路配置文件, 通过 `vendor/rockchip/modules/audio_aidl/config/platform/audio_rockchip_platform_config.xml` 确认, 没有匹配则使用默认的mixer_paths_default.xml

比如 2 channel mic + 2 channel 回采

```
diff --git a/config/mixer-path/mixer_paths_default.xml b/config/mixer-
path/mixer_paths_default.xml
index 23a7860..6c7b5b0 100644
--- a/config/mixer-path/mixer_paths_default.xml
+++ b/config/mixer-path/mixer_paths_default.xml
@@ -35,6 +35,7 @@
     <!-- Out Speaker -->
     <path name="out_speaker normal">
         <ctl name="Playback Path" value="SPK"/>
+        <ctl name="SAI1 Software Digital Loopback Mode" value="4CH: 2 Mics + 2
Loopbacks"/>
     </path>
     <path name="out_speaker in_call">
         <ctl name="Playback Path" value="SPK"/>
@@ -117,6 +118,7 @@
     <!-- In Builtin MIC -->
     <path name="in_builtin_mic normal">
         <ctl name="Capture MIC Path" value="Main Mic"/>
+        <ctl name="SAI1 Software Digital Loopback Mode" value="4CH: 2 Mics + 2
Loopbacks"/>
     </path>

     <!-- In Wired Headset -->
```

总共有以下几个选项, 根据对应的 MIC 声道数和回采声道数配置即可，可以tinymix查看

```
static const char *const dlp_text[] = {
        "Disabled",
        "2CH: 1 Loopback + 1 Mic",
        "2CH: 1 Mic + 1 Loopback",
        "2CH: 1 Mic + 1 Loopback-mixed",
        "2CH: 2 Loopbacks",
        "4CH: 2 Mics + 2 Loopbacks",
        "4CH: 2 Mics + 1 Loopback-mixed",
        "4CH: 4 Loopbacks",
        "6CH: 4 Mics + 2 Loopbacks",
        "6CH: 4 Mics + 1 Loopback-mixed",
        "6CH: 6 Loopbacks",
        "8CH: 6 Mics + 2 Loopbacks",
        "8CH: 6 Mics + 1 Loopback-mixed",
        "8CH: 8 Loopbacks",
        "10CH: 8 Mics + 2 Loopbacks",
        "10CH: 8 Mics + 1 Loopback-mixed",
        "16CH: 8 Mics + 8 Loopbacks",
};
```

调整音频参数

修改声道参数

回采配置完成后，修改最终的录音通道数和回采通道数，SDK默认2mic + 0回采

修改 `vendor/rockchip/modules/audio_aidl/modules/preprocess/libs/include/rkaudio_preprocess.h` 的声道参数, 如下：

参数	说明
NUM_CHANNEL 宏	总的声道数, 比如 2 MIC + 2 回采, 则为 4
NUM_REF_CHANNEL 宏	回采声道数, 比如 2 回采, 则为 2
static float Array[NUM_CHANNEL]	声道排序, 从 0 开始, 比如 2 MIC + 2 回采, 如果没有乱序则为 `{0, 1, 2, 3}`

若使能bf模块，需要修改rkaudio_effect_preprocess.json中的mic通道数`num_channel`

```
"bf": {
            "status": "enable",
            ....
            "num_channel": 2,
            "param_path":
"/vendor/etc/wopt_2mic_linear_40mm_5inputbeam_5outputbeam.bin",
            "license_path": "/data/key.lic"
        },
```

修改 MIC 录音参数

需要设置基本的参数以满足算法库的初始化

算法库要求 MIC 录音的帧长为 16ms，向下取整

```
period_size = (int)(rate * 0.016 / 4) * 4
```

比如采样率 48000, `period_size = (int)(48000 * 0.016 / 4) * 4 = 768`

比如采样率 44100, `period_size = (int)(44100 * 0.016 / 4) * 4 = 704`

默认算法录音参数定义位于`audio_aidl/modules/base/StreamPrimary.cpp`顶部

```
static struct pcm_config config_algo = {
    // ...
    .rate = 48000,
    ...
};
```

period_size会根据latency自动计算，默认latency即为16ms，无需修改，定义在`audio_aidl/modules/primary/ModulePrimary.cpp`中

```
int32_t ModulePrimary::getNominalLatencyMs(const AudioPortConfig& portConfig) {
    bool isInput = portConfig.flags.value().getTag() == AudioIoFlags::Tag::input;
    int32_t latencyMs = isInput ? 16 : 20;
}
```

单 MIC 目前是以 2 channel 来处理, 只取其中一个有效的声道, 忽略另一个声道

验证

可以通过dump算法输入和输出的原始数据来确认效果和分析问题

设置为 permissive 模式和开启 rkpreprocess 调试开关, 设备运行期间执行一次即可

```
setenforce 0
setprop vendor.audio.preproc.debug 1
```

重新开始语音或录音一段时间后, 会在 `/data/misc/audioserver/` 目录生成输入和输出文件, 导出并用 audacity 或其他 PC 工具分析这两个文件

```
adb pull /data/misc/audioserver/preproc_input_xxxhz_xxxch.pcm
adb pull /data/misc/audioserver/preproc_output_xxxhz_xxxch.pcm
```

其中 xxxhz 表示采样率, xxxch 表示声道数

`输入文件`表示算法的输入数据流, 而`输出文件`则表示算法的输出

使用 audacity 分析时, 依次点击`文件`, `导入`, `原始数据`, 然后修改采样率和声道数即可

确认符合效果之后把 rkaudio_effect_preprocess.json 更新到 `vendor/rockchip/modules/audio_aidl/modules/preprocess/`

Q&A

不清楚tinyalsa工具如何使用的参考文档，也可自行上网查询资料

如何确认需要启用哪些算法模块

1. 如果需要回声消除, 确保输入的数据包含回采数据, 需要启用 aec, bf, fast_aec, aes
2. 如果需要降噪, 需要启用 bf, anr
3. 其他需求请参考文档的介绍并确认

没有生成输出文件

如果算法库初始化失败

```
12-20 09:03:33.753 442 2298 D preproc : preproc_fx_process: failed to initialize algorithm
12-20 09:03:33.755 442 2298 D preproc-algo: algo_init: failed to initialize algorithm
```

1. 如果是付费版本, 确认是否授权成功, 参考授权小节
2. 确认是否正确设置音频参数

输出文件数据全 0

确认另一个输入文件的数据是否全 0, 如果也全 0, 确认下面两项

- 确认声卡至 audio_hal 数据是否正常

```
# tinycap /sdcard/record.wav -D xxx -c xxx -r xxx
```

tinycap 参数根据实际的声卡序号, 声道数以及采样率填上, 之后导出录音文件 record.wav 并分析

- 确认是否正确修改 MIC 录音帧长, 请参考对应的章节重新确认

如果在启用某个算法库的模块时出现这个问题, 则说明这个模块在当前版本的算法库不受支持, 请关闭模块或者申请受支持的算法库版本

cannot set hw params

这表示无法以指定的声道 (MIC + 回采) 录音, 以 2 channel MIC 和 2 channel 回采为例

```
# tinycap /sdcard/mic.wav -D xxx -c 4 -r 48000
Unable to open PCM device (cannot set hw params: Invalid argument)
Captured 0 frames
```

> xxx 是 cat /proc/asound/cards 后看到的对应的声卡序号

检查通路设置

若加入 DLP 后，按以下两点顺序排查

1. 确认是否能正常设置通路

```
# tinymix -D xxx "Software Digital Loopback Mode" "4CH: 2 Mics + 2 Loopbacks"
```

如果报错了需要确以下 3 点

```
Invalid mixer control: Software Digital Loopback Mode
```

- 需要确认当前版本的 kernel 是否增加了 DLP 支持, 没有则更新 SDK 或加入补丁

```
# grep rockchip,digital-loopback -r ./sound/soc/rockchip/
./sound/soc/rockchip/rockchip_sai.c:    if (device_property_read_bool(&pdev->dev,
"rockchip,digital-loopback"))
./sound/soc/rockchip/rockchip_i2s.c:    if (device_property_read_bool(&pdev->dev,
"rockchip,digital-loopback"))
./sound/soc/rockchip/rockchip_i2s_tdm.c:        if (device_property_read_bool(dev,
"rockchip,digital-loopback")) {
```

- 需要确认 kernel 是否开启了 DLP

```
CONFIG_SND_SOC_ROCKCHIP_DLP_PCM
```

- 需要在 dts 对应的 cpu dai 上使能 DLP

```
&i2s {
        rockchip,digital-loopback;
};
```

> cpu dai 可以是: i2s0_8ch, i2s1_8ch, i2s2_2ch, i2s3_2ch, sai0, sai1 等等

2. 确认通路是否生效

   录音时执行

```
# tinymix -D xxx
...
13      ENUM    1       Software Digital Loopback Mode          4CH: 2 Mics + 2
Loopbacks
...
```

检查声卡相关

直接使用tinypcminfo查看声卡能力

```
rk3576_a16:/ # tinypcminfo -D 0
Info for card 0, device 0:

```

```
PCM out:
      Access:   0x000009
   Format[0]:   0x000044
   Format[1]:   00000000
 Format Name:   S16_LE, S24_LE
   Subformat:   0x000003
        Rate:   min=8000Hz      max=96000Hz
    Channels:   min=1           max=2
 Sample bits:   min=16          max=32
 Period size:   min=32          max=131072
Period count:   min=2           max=8192

PCM in:
      Access:   0x000009
   Format[0]:   0x000044
   Format[1]:   00000000
 Format Name:   S16_LE, S24_LE
   Subformat:   0x000003
        Rate:   min=8000Hz      max=96000Hz
    Channels:   min=1           max=2
 Sample bits:   min=16          max=32
 Period size:   min=8           max=131072
Period count:   min=2           max=32768
```

这里pcm in的channels显示max=2，表示最大只支持2 channel capture

在codec驱动中应该可以看到channel被限制

```
struct snd_soc_dai_driver demo_dai = {
    // 没有 capture, 或者 channels_max 小于 4 channel, 则说明不支持
    .capture = {
        .channels_min = 1,
        .channels_max = 2,
    },
    // ...
};
```

如果不支持则可以使用 "rockchip,multicodecs-card", 同时绑定一个辅助的虚拟 codec

```
// 修改前 (例1)
#if 0
demo_sound: demo-sound {
    status = "okay";
    compatible = "simple-audio-card";
    simple-audio-card,format = "i2s";
    simple-audio-card,name = "rockchip,demo-codec";
    simple-audio-card,mclk-fs = <256>;
    simple-audio-card,cpu {
        sound-dai = <&i2s>;
    };
    simple-audio-card,codec {
        sound-dai = <&demo>;
```

```
    };
};
#endif

// 修改前 (例2)
#if 0
demo_sound: demo-sound {
    status = "okay";
    compatible = "rockchip,multicodecs-card";
    rockchip,format = "i2s";
    rockchip,card-name = "rockchip,demo-codec";
    rockchip,mclk-fs = <256>;
    rockchip,cpu = <&i2s>;
    rockchip,codec = <&demo>;
};
#endif
```

```
// 修改后
// 声明虚拟 codec 节点
demo_aux: demo-aux {
    status = "okay";
    compatible = "rockchip,dummy-codec";
    sound-name-prefix = "demo-aux";
    #sound-dai-cells = <0>;
};

// 使用 "rockchip,multicodecs-card" 绑定 codec 和虚拟 codec
demo_sound: demo-sound {
    status = "okay";
    compatible = "rockchip,multicodecs-card";
    rockchip,format = "i2s";
    rockchip,card-name = "rockchip,demo-codec";
    rockchip,mclk-fs = <256>;
    rockchip,cpu = <&i2s>;
    rockchip,codec = <&demo &demo_aux>;
};
```

或者可以修改codec驱动中的channels_max值

```
struct snd_soc_dai_driver demo_dai = {
    .capture = {
        .channels_min = 1,
        .channels_max = 4, // 由2改为4
    },
    // ...
};
```

示例

以rk3576-tablet -v11开发板为例

确定需求

需要开AEC进行回声消除-------->启用aec相关模块

```
diff --git a/modules/preprocess/rkaudio_effect_preprocess.json
b/modules/preprocess/rkaudio_effect_preprocess.json
index 95738fd..716c173 100644
--- a/modules/preprocess/rkaudio_effect_preprocess.json
+++ b/modules/preprocess/rkaudio_effect_preprocess.json
@@ -1,7 +1,7 @@
 {
   "skv_configs": {
     "aec": {
-      "status" : "disable",
+      "status" : "enable",
       "drop_ref_channel" : 0,
       "model_aec_en" : "disable",
       "delay_len": 0,
@@ -26,10 +26,10 @@
       "drop_ref_channel" : 0
     },
     "fast_aec": {
-      "status" : "disable"
+      "status" : "enable"
     },
     "aes": {
-      "status": "disable",
+      "status": "enable",
       "beta_up": "float:0.003",
       "beta_down": "float:0.003",
       "beta_up_low": "float:0.003",
```

确认硬件配置

cat /proc/asound/cards查看声卡-------->确认mic和speaker配置到es8388声卡

```
rk3576_a16:/ # cat /proc/asound/cards
 0 [rockchipes8388 ]: rockchip-es8388 - rockchip-es8388
                      rockchip-es8388
 1 [rockchipdp0    ]: rockchip-dp0 - rockchip-dp0
                      rockchip-dp0
```

查看原理图-------->无硬件回采

查看dts-------->cpu dai使用的是sai，可以使用SAI dataline 间路由/DLP

```
es8388_sound: es8388-sound {
...
 rockchip,cpu = <&sai1>;
 rockchip,codec = <&es8388>;
...
 };
```

配置回采方式

cpu dai使用的是sai，可以使用SAI dataline 间路由/DLP，二者选择其一即可

SAI dataline 间路由：

```
diff --git a/config/mixer-path/mixer_paths_es8388.xml b/config/mixer-
path/mixer_paths_es8388.xml
index 5056929..93b760e 100644
--- a/config/mixer-path/mixer_paths_es8388.xml
+++ b/config/mixer-path/mixer_paths_es8388.xml
@@ -107,6 +108,8 @@
         <ctl name="ALC Capture Function" value="Stereo"/>
         <ctl name="ALC Capture Max PGA" value="5"/>
         <ctl name="ADC Data Select" value="Left Left"/>
+        <ctl name="SAI1 SDI1 Loopback Src Select" value="From SDO0"/>
+        <ctl name="SAI1 SDI1 Loopback Switch" value="Enable"/>
     </path>

     <!-- In Wired Headset -->
```

DLP方式：

确认CONFIG_SND_SOC_ROCKCHIP_DLP_PCM是否打开，不打开则需要手动添加

```
rk:~/RK_android16/kernel-6.12$ grep "CONFIG_SND_SOC_ROCKCHIP_DLP_PCM" .config
CONFIG_SND_SOC_ROCKCHIP_DLP_PCM=y
```

在内核dts打开DLP开关

```
diff --git a/arch/arm64/boot/dts/rockchip/rk3576-tablet.dtsi
b/arch/arm64/boot/dts/rockchip/rk3576-tablet.dtsi
index e699f88d03f54..94e9b5d9550ce 100644
--- a/arch/arm64/boot/dts/rockchip/rk3576-tablet.dtsi
+++ b/arch/arm64/boot/dts/rockchip/rk3576-tablet.dtsi
@@ -1115,6 +1115,7 @@ &route_dsi {

 &sai1 {
        status = "okay";
+       rockchip,digital-loopback;
        pinctrl-names = "default";
        pinctrl-0 = <&sai1m0_lrck
                     &sai1m0_sclk
```

修改对应的通路配置文件

```
diff --git a/config/mixer-path/mixer_paths_es8388.xml b/config/mixer-path/mixer_paths_es8388.xml
index 5056929..e594a70 100644
--- a/config/mixer-path/mixer_paths_es8388.xml
+++ b/config/mixer-path/mixer_paths_es8388.xml
@@ -50,6 +50,7 @@
         <ctl name="aw87xxx_profile_switch_1" value="Music"/>
         <ctl name="OUT2 Switch" value="1"/>
         <ctl name="Speaker Switch" value="1"/>
+        <ctl name="SAI1 Software Digital Loopback Mode" value="4CH: 2 Mics + 2 Loopbacks"/>
     </path>
     <path name="out_speaker ringtone"/>
     <path name="out_speaker in_call"/>
@@ -107,6 +108,7 @@
         <ctl name="ALC Capture Function" value="Stereo"/>
         <ctl name="ALC Capture Max PGA" value="5"/>
         <ctl name="ADC Data Select" value="Left Left"/>
+        <ctl name="SAI1 Software Digital Loopback Mode" value="4CH: 2 Mics + 2 Loopbacks"/>
     </path>

     <!-- In Wired Headset -->
```

修改音频参数

默认为2mic + 0回采配置，若要配置2mic + 2回采

```
diff --git a/modules/preprocess/libs/include/rkaudio_preprocess.h b/modules/preprocess/libs/include/rkaudio_preprocess.h
index 8e6c532..a974fbc 100644
--- a/modules/preprocess/libs/include/rkaudio_preprocess.h
+++ b/modules/preprocess/libs/include/rkaudio_preprocess.h
@@ -7,14 +7,14 @@ extern "C" {
 #endif
 //#define ENABLE_STUDIO 1

-#define NUM_CHANNEL                                 2
-#define NUM_REF_CHANNEL             0
+#define NUM_CHANNEL                                 4
+#define NUM_REF_CHANNEL             2
 #define NUM_SRC_CHANNEL               (NUM_CHANNEL - NUM_REF_CHANNEL)
 #define NUM_DROP_CHANNEL            0
 #define REF_POSITION                                1

-       static float Array[NUM_CHANNEL] = { 0, 1 };
-       // static float Array[NUM_CHANNEL] = { 0, 1, 2, 3 };
+       // static float Array[NUM_CHANNEL] = { 0, 1 };
+       static float Array[NUM_CHANNEL] = { 0, 1, 2, 3 }
```

pcm_config仍然使用SDK默认

```
static struct pcm_config config_algo = {
    // ...
    .rate = 48000,
    ...
};
```

验证分析

参考这里验证算法输出是否符合预期